PjBL: Autenticação JWT

Segurança da Informação — Prof. Mateus Nunes

Desafio: Criar uma API em Java Nativo que realize o fluxo completo de autenticação (Cadastro, Login e Proteção de Rotas) utilizando Tokens JWT e persistência em JSON.

Regras: Para a resolução deste trabalho, os alunos devem se organizar em duplas, haverá defesa do código e o trabalho entregue será submetido a uma rúbrica de avaliação.

Entregável: Código AVA + GitHub + Relatório PDF.

Rúbrica de Avaliação

Critério	Descrição	Peso
Segurança	Uso de BCrypt para hashes e chaves via variáveis de ambiente.	2.5
Lógica JWT	Geração, validação de assinatura e controle de expiração.	2.5
Persistência	Manipulação correta do JSON (findAll/save) e unicidade de e-mail.	2.5
Integração	Funcionamento pleno com o Frontend fornecido (Login/Dashboard).	2.5
Defesa (Critério multiplicador CM)	Domínio técnico e explicação clara durante a arguição oral.	(0 a 1.0)
TOTAL		**nota x CM**

Penalidade Crítica

Duplas com API Java iguais terão ambos os trabalhos zerados.
Senhas em texto claro ou aceitação de alg:none resultam em nota **ZERO** no critério de Segurança.

1. Apresentação e Arquitetura

Este projeto propõe o desenvolvimento de uma API de Autenticação utilizando **Java SE Nativo** (sem frameworks). O objetivo é que você compreenda como os dados trafegam desde a chegada de uma requisição HTTP até a persistência segura no disco.

1.1. O Papel do Frontend Fornecido

Para que o foco do aprendizado seja a **Lógica de Segurança no Back-end**, você receberá um **Frontend completo** (HTML/JS).

- O frontend está configurado para se comunicar com `http://localhost:8080`.
- Ele utiliza um padrão de segurança em memória (IIFE) para armazenar o JWT, evitando ataques de roubo de sessão via *LocalStorage*.
- Sua missão é construir o motor (API) que responda corretamente a essa interface.

1.2. Estrutura do Back-end

Diferente de frameworks modernos que escondem a complexidade, aqui você trabalhará com as camadas explícitas:

1. **Handlers (Controller):** Onde o protocolo HTTP é tratado "na mão"(`HttpExchange`).
2. **Repository:** Camada de persistência que manipula o arquivo `usuarios.json`.
3. **Service:** Onde reside a inteligência criptográfica do JWT.

2. O Desafio Técnico

Você recebeu um esqueleto de projeto. Sua missão é conectar essas peças e garantir que nenhum dado sensível seja exposto ou armazenado de forma insegura.

2.1. Tarefas de Implementação

</> 1. Repositório de Dados (`UsuarioRepository.java`)

Implementar a persistência utilizando a biblioteca **Jackson**.

- **findAll/save:** Ler e escrever a lista de usuários no arquivo JSON.
- **Constraint de Unicidade:** O sistema deve impedir, via código, o cadastro de dois usuários com o mesmo e-mail (ignorar maiúsculas/minúsculas).

</> 2. Segurança de Senhas (`AuthHandler.java`)

Implementar a proteção contra vazamento de credenciais.

- **Hashing:** Use BCrypt com fator de custo 12.

- **Verificação:** No login, utilize `BCrypt.checkpw()` para comparar a senha vinda do JSON com a digitada pelo usuário. Nunca use `.equals()` para senhas.

3. Ciclo de Vida do JWT (JwtService.java)

Implementar o padrão RFC 7519 para garantir a integridade da sessão.

- **Geração:** Criar o token com as claims de e-mail e cargo (*role*).
- **Validação:** Verificar a assinatura do token em cada requisição protegida e validar o tempo de expiração (*exp*).
- **Segredo:** A `SECRET_KEY` deve ser carregada via variável de ambiente por segurança.

3. Relatório de Fixação (Manuscrito)

Instruções

As perguntas abaixo devem ser respondidas à mão, com letra legível, para entrega junto ao projeto. Este relatório visa avaliar a compreensão teórica da equipe sobre o que foi implementado.

Pergunta 1: Por que não podemos utilizar o método `String.equals()` para comparar a senha enviada pelo usuário com a senha armazenada no arquivo JSON? Explique o papel do BCrypt neste processo.

R: Nao devemos usar `String.equals()` porque a senha nao deve ser salva em texto claro. Se comparasse com `equals`, significaria que a senha do arquivo tambem estaria pura, o que e inseguro. O BCrypt gera um hash com salt, entao mesmo duas senhas iguais podem gerar hashes diferentes. No login usamos `BCrypt.checkpw`, que sabe comparar a senha digitada com o hash salvo.

Pergunta 2: Durante a implementação do `JwtService`, você utilizou uma `SECRET_KEY`. Qual a consequência para a segurança do sistema caso um atacante tenha acesso a essa chave?

R: Se um atacante descobrir a `SECRET_KEY`, ele pode criar tokens falsos e assinar esses tokens como se fossem legitimos. Isso permitiria acessar rotas protegidas e ate colocar outro e-mail ou role no payload. Por isso a chave fica em variavel de ambiente, fora do codigo.

Pergunta 3: O que é o ataque de "Enumeração de Usuários"e como a sua implementação no `AuthHandler` (mensagens de erro de login) ajuda a mitigar este risco?

R: Enumeracao de usuarios e quando o atacante tenta descobrir quais e-mails existem no sistema. Se o login respondesse "e-mail nao encontrado", ele conseguiria montar uma lista de usuarios validos. A implementacao retorna sempre "E-mail ou senha invalidos", entao nao revela qual parte esta errada. _

Pergunta 4: No JWT, qual a diferença entre *codificação* (Base64) e *assinatura* (HMAC)? Por que é perigoso armazenar uma senha no Payload do token?

R: Base64 e apenas codificacao: qualquer pessoa consegue decodificar e ler o header e o payload do JWT. Assinatura HMAC e a parte que garante que o token nao foi alterado, porque ela depende da chave secreta. E perigoso colocar senha no payload porque o payload nao e criptografado, so codificado.

Pergunta 5: Por que o armazenamento do token no Frontend foi feito exclusivamente em memória (IIFE) em vez de `localStorage`? Qual ataque estamos tentando evitar?

R: Guardar token em memoria reduz o risco de roubo por XSS. Se o token ficasse no `localStorage`, um script malicioso rodando na pagina poderia ler e enviar esse token para um atacante. Em memoria, o token some ao fechar/atualizar o fluxo e nao fica persistido no navegador.

Assinatura dos Integrantes: